Agentic Reasoning Strategies: From ReAct to Multi-Agent Systems

Explore advanced agentic reasoning strategies such as the ReAct framework, self-correction, plan-and-execute, and multi-agent collaboration. Understand how these methods enable agents to think, adapt, and coordinate complex tasks for improved accuracy and efficiency.

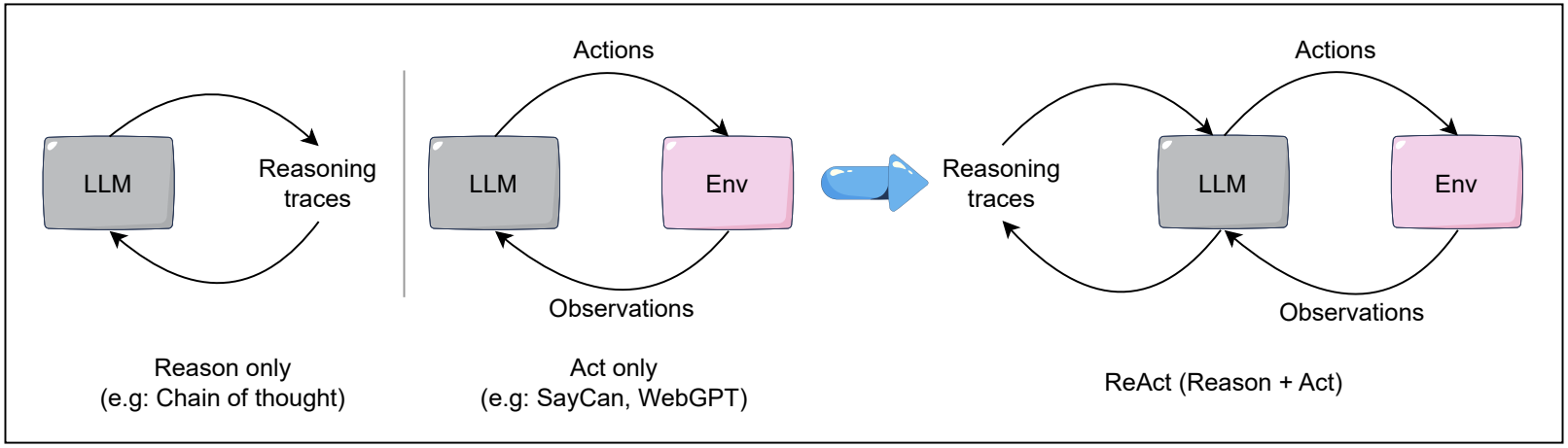
In our previous lesson, we disassembled an agent into its four core components: the brain, the tools, the planner, and memory. We established that the “planner” is the agent’s operating system, responsible for orchestrating the entire problem-solving process.

Now that we understand the parts, it’s time to learn about the advanced strategies the agent uses to think. In this lesson, we will explore the powerful reasoning frameworks that power an agent’s responses. One of the most influential and foundational strategies we’ll explore is called ReAct. It’s the basis of nearly every modern reasoning loop you’ll encounter, from ChatGPT’s tool use to LlamaIndex’s agent orchestration, making it the perfect starting point for our deep dive.

Ask

What is ReAct?

ReAct stands for **Reason + Act**. It's a specific prompting strategy that forces an LLM to "think out loud" before it acts. This verbalization of its reasoning process makes the agent's behavior more transparent and reliable, forming the foundation of modern, effective agentic systems.



ReAct creates a loop where reasoning (Reasoning traces) informs actions, and observations from those actions inform the next reasoning step

The engine room: The ReAct prompt template

The agent's ability to reason isn't miraculous; it's meticulously guided by a master prompt, often called a **prompt template**. Before the agent even sees your query, it has been given a complex set of instructions that act as its operating manual.

This master prompt typically contains four key pieces of information.

Ask

1. **The persona and goal:** A high-level instruction defining its role, like, “You are an expert AI research assistant. Your goal is to help users by answering their questions accurately and methodically.”
2. **The tool listing:** A formatted list of all the tools the agent has access to, including their names and, most importantly, the descriptions we write for them. This is how the agent knows what it’s capable of.
3. **The formatting instructions:** Explicit instructions on how to respond. It is told to use the exact thought, action, and action input: syntax. This structured format is what allows the framework to parse its response and execute the correct tool.
4. **Memory (the scratchpad):** The history of the current conversation, including all of the previous thought -> act -> observe cycles.

Ask

The structure of a ReAct prompt template, which combines the agent's role, its available tools, formatting rules, and the conversation history into a single context for the LLM

When we send a query, it is appended to this large prompt, giving the LLM all the context it needs to decide on its first thought.

A detailed trace of the ReAct cycle

Let's revisit our example with this new depth. Imagine we are looking at the raw, unfiltered output from the agent's "brain" (the LLM).

Ask

> **User query:** “What is a Transformer, and who are its main authors?”

Cycle 1: Defining the core concept

- **Agent’s internal monologue:** The agent receives the query and its master prompt. It analyzes the query against its available tools.
- **LLM raw output (the trace):**

```
Thought: The user has a two-part question. I should address the first part: defining "Transformer". The `defir
Action: definition_tool [cite: 128]
Action Input: {"term": "Transformer Architecture"} [cite: 128]
```

- **Framework action:** The LlamaIndex framework parses this text. It sees the `Action` and `Action Input` and calls the corresponding Python function: `define_term_function(term="Transformer Architecture")`.
- **Observation:** The agent receives the return value from the function.

```
Observation: The Transformer in deep learning is an architecture that relies on the self-attention mechanism t
```

Cycle 2: Finding the authors

- **Agent’s internal monologue:** The agent now takes the `Observation` from Cycle 1 and appends it to its memory (the scratchpad). It re-evaluates its progress.
- **LLM raw output (the trace):**

Ask

```
Thought: I have successfully defined the term "Transformer". Now I need to find the authors. The user's local
Action: local_rag_tool [cite: 132]
Action Input: {"query": "Who are the authors of the paper 'Attention Is All You Need'?"} [cite: 132]
```

- **Framework action:** The framework again parses the output and calls the RAG tool: `local_rag_tool.call(query="Who are the authors of the paper 'Attention Is All You Need'?")`.
- **Observation:** The agent receives the list of authors retrieved from the PDF.

```
Observation: The authors of 'Attention Is All You Need' are Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob U
```

Final synthesis

- **LLM raw output (the trace):**

```
Thought: I have successfully gathered both pieces of information requested by the user. I have the definition
```

- **Final answer (streamed to user):**

> **Query reply:** The Transformer is a deep learning architecture that relies on the self-attention mechanism. The main authors of the foundational paper, “Attention Is All You Need,” are Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, and Lukasz Kaiser.

Ask

Notice how the agent made a strategic choice. It first used the `definition_tool` for general knowledge and then used the specialized `local_rag_tool` to query its private, trusted documents. This ability to intelligently switch

between general information and high-fidelity retrieved information, is a hallmark of a sophisticated agentic RAG system.

Why this level of detail is so powerful

Understanding this deeper process reveals why ReAct is so effective.

- **Debuggability:** The raw trace shows you exactly what the LLM is thinking and what it's trying to do. If it fails, the error is almost always visible in this trace.
- **Reduced hallucination:** The agent is forced to ground its reasoning in a concrete observation from a tool before proceeding. It talks to itself about the data it receives, rather than just its internal knowledge.
- **Adaptability:** The agent can dynamically change its plan based on the observation. If a tool had returned an error, the next thought would be about how to recover from that specific error.

Here's a question that could come to our mind at this point.

1. Couldn't a powerful LLM just answer this directly?

Show Answer

Did you find this helpful?

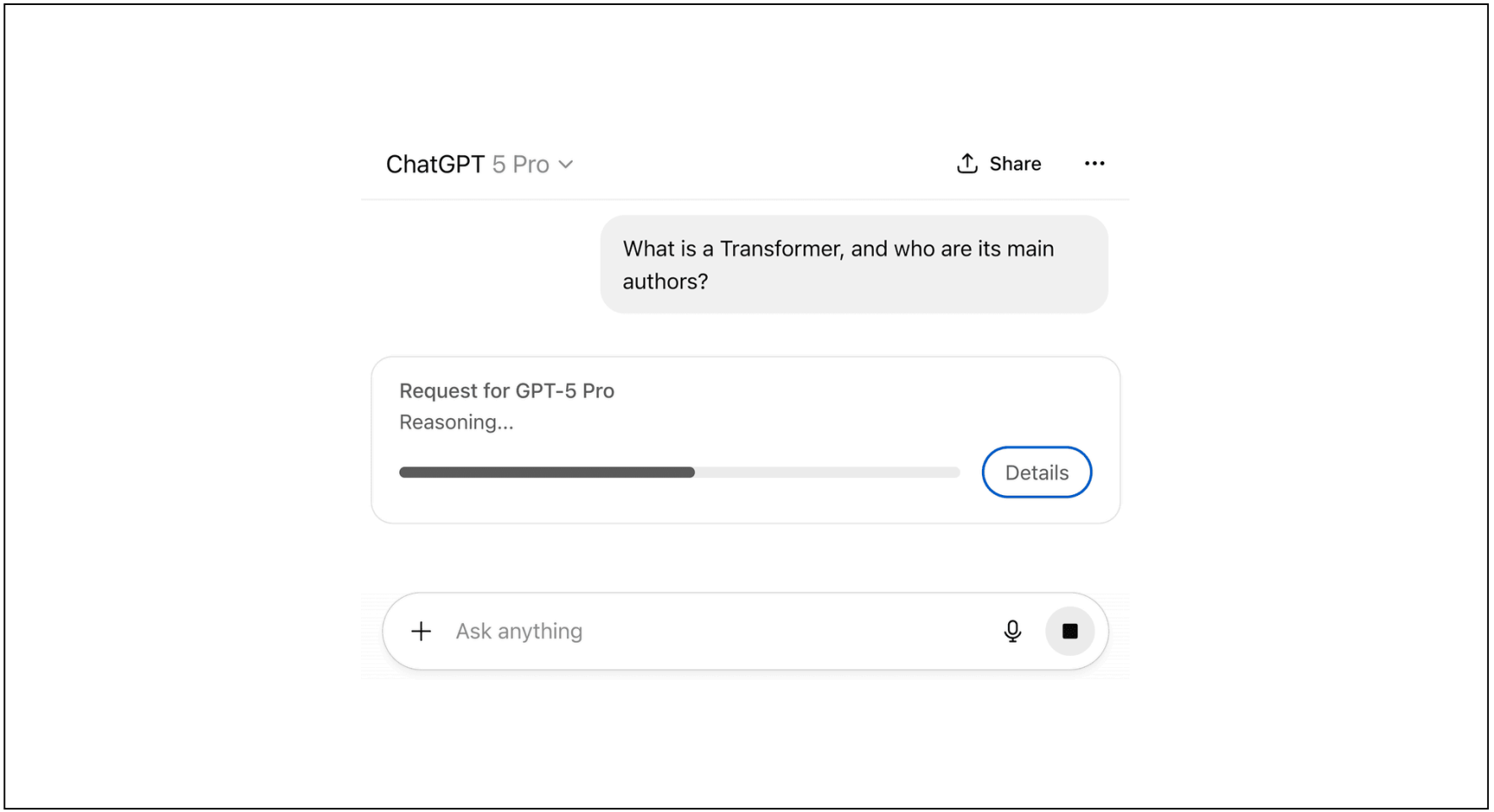
Ask

The industry standard: ReAct in the wild

The `Thought -> Act` reasoning cycle isn't just an academic concept; it's the fundamental principle behind the "agentic" capabilities of the world's leading AI models.

- When you use a feature like OpenAI's "Research" mode in ChatGPT, you are witnessing this loop in action. If you expand the details, you can often see the model's internal reasoning as it decides to browse the web (an action), reads the results (an observation), and then synthesizes an answer. This mirrors the ReAct cycle of reasoning, acting, and observing.

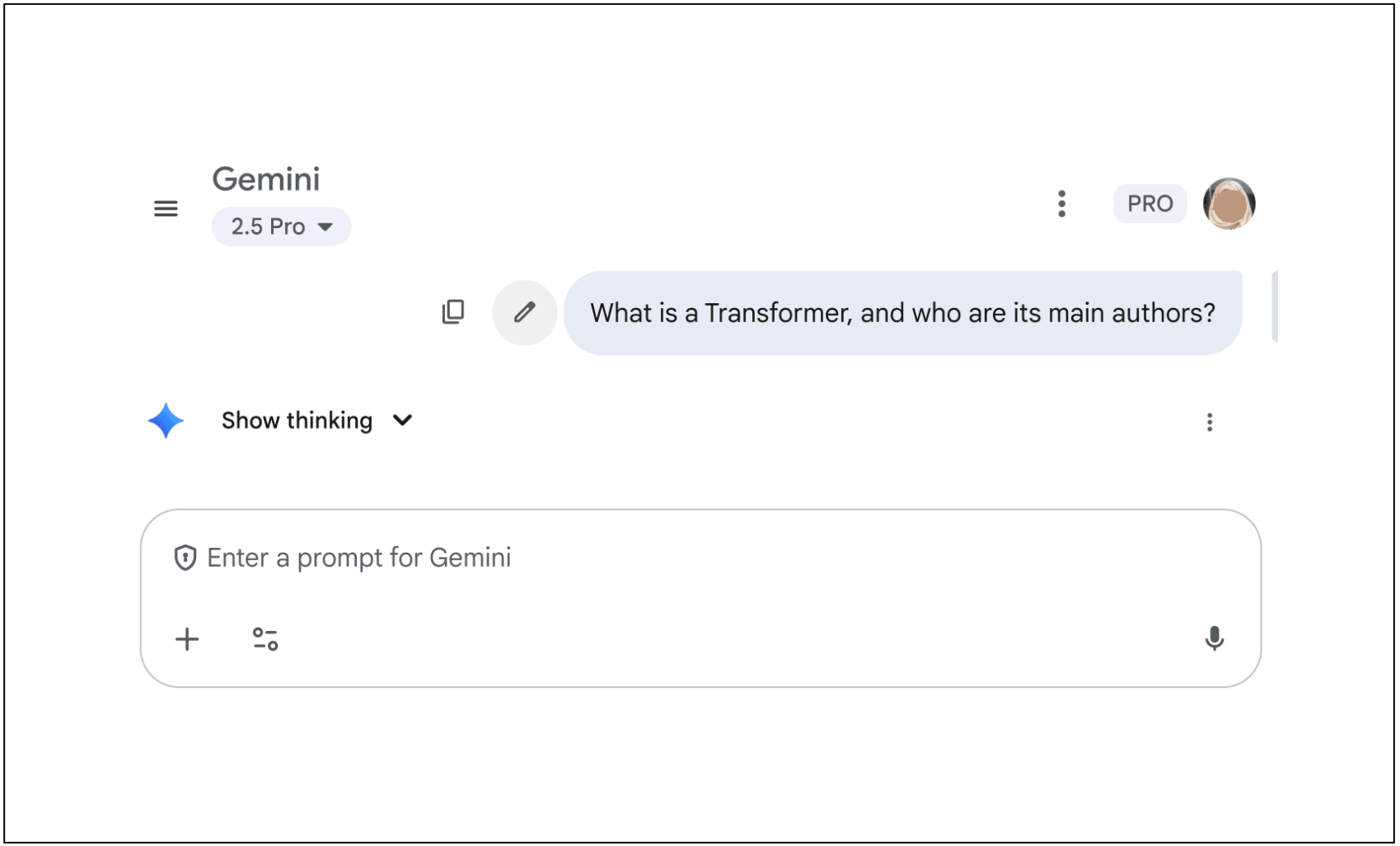
Ask



1 / 2

- Similarly, Google's Gemini models have built-in tool use (function calling). When it decides to use a tool, it is performing the act step of the cycle. The API response it gets back is the observation, which it then u its next thought step to formulate the final answer.

Ask



1 / 3

By learning the ReAct framework, we are learning the core, universal logic that powers modern, useful AI. skills we build here are directly applicable to building with any major large language model.

Ask

Beyond ReAct: Other reasoning patterns

The ReAct framework is a powerful and versatile strategy, but it’s not the only way an agent can reason. As we become more advanced agent architects, we’ll encounter other patterns designed for specific tasks. Let’s explore two important ones.

Self-correction and reflection

This is a more advanced pattern where an agent is programmed to critique its own work before providing a final answer. It adds a layer of quality control to the agent’s output. The process involves at least two steps.

Ask

The self-correction or reflection loop. The agent first generates an initial output, then uses a second LLM call to “reflect” on and critique that output, leading to a more refined and accurate final response

1. **Initial generation:** The agent completes a task, for example, writing a summary of a research paper.
2. **Critique and refinement:** Instead of immediately returning the summary, the agent makes a second LLM call. It feeds its own summary into a new prompt that asks it to act as a critic, with a prompt like, “Here is a summary I wrote. Is it accurate? Does it miss any key details? How can it be improved?.” The agent then refines its initial output based on the critique it generated for itself.

This is like an agent writing a first draft and then taking the time to proofread and edit its own work before submitting it. This pattern can significantly increase the quality and factual accuracy of an agent’s final response.

> This strategy is inspired by research on ‘Reflexion’ agents, where an LLM uses a self-evaluation step to improve factual accuracy and reasoning consistency.

Imagine our agent uses the RAG tool to answer, “What was the company’s revenue in Q3?.”

1. **Initial generation:** It retrieves several text chunks and generates an answer, “The company’s revenue was $50 million, showing strong growth.”
2. **Self-correction step:** The agent then makes a second call to the LLM with a “critique” prompt, “Here are the retrieved text chunks: [insert chunks here]. Here is the answer I generated: [insert answer here]. Does the provided context explicitly state that the growth was ‘strong’? Or is that an inference I made?”
3. **Refinement:** The LLM, acting as a critic, might respond, “The context states the revenue figure but does not use the word ‘strong.’” The agent then refines its answer to be more factually grounded, “The company’s

Ask

revenue was $50 million.”

Plan-and-execute

This is an alternative to the iterative, step-by-step ReAct model. As the name suggests, it separates the planning phase from the execution phase. It works like this:

Ask

The plan-and-execute workflow. This is where the agent first generates a complete, multi-step task list. It then executes all tasks sequentially, only replanning if necessary after the initial execution phase is complete

1. **Planning:** The agent first analyzes the user's query and creates a complete, multi-step plan from start to finish. It lists out every tool call it intends to make.
2. **Execution:** Once the plan is finalized, the agent executes all the steps in the plan sequentially without stopping to re-evaluate or change course.

> **ReAct vs. plan-and-execute:** These two strategies have distinct advantages and are suited for different kinds of tasks.
>
> - ReAct is better for complex, unpredictable tasks where the plan might need to change based on new information. Its ability to adapt after each observe step makes it more robust for research and exploration.
> - Plan-and-execute can be faster and more efficient for simple, well-defined tasks where the workflow is predictable. For example, if a task is always "Fetch data from API A, then process it with function B," a pre-defined plan can be more direct.
>
> In essence, plan-and-execute trades adaptability for efficiency. ReAct reasons after every step, while plan-and-execute reasons once at the start.

A task like "Generate a quarterly performance summary by comparing the Q1 and Q2 financial report PDFs" is well-suited for plan-and-execute.

An agent could generate this predictable plan up-front,

Ask

1. **Step A:** Use the `local_rag_tool` to find the 'Net Revenue' from the `Q1_report.pdf`.
2. **Step B:** Use the `local_rag_tool` to find the 'Net Revenue' from the `Q2_report.pdf`.
3. **Step C:** Pass the results of Step A and B to a `calculator_tool` to find the percentage change.

4. **Step D:** Synthesize the findings into a summary.

Since the steps are known in advance, the agent can create a full plan before executing it.

The strategies we've discussed so far, ReAct, self-reflection, and plan-and-execute, all operate within the context of a single agent making decisions. However, just as in human organizations, some problems are too complex for one individual. This leads us to the next frontier of agentic design: creating systems where multiple specialized agents collaborate to achieve a common goal.

Multi-agent systems: A team of specialists

So far, we have focused on a single agent completing a task. The next logical and exciting frontier is to create systems where multiple specialized agents collaborate to achieve a common goal. The concept behind multi-agent systems is that a team of specialized agents working together is often more effective than a single, generalist agent trying to do everything. Let's think of this using an architectural firm analogy.

To design and build a skyscraper, a firm brings together a team of diverse experts. A single person cannot possibly handle the complexity. We can model our most advanced agentic systems in the same way.

Ask

A typical multi-agent architecture . A central “Manager” agent receives a task and delegates sub-tasks to specialized “Worker” agents, each equipped with its own set of tools. This allows for a modular and highly specialized approach to problem-solving

- A lead architect agent acts as the project manager. It meets with the client to understand their vision (the user’s query), creates the high-level conceptual design, and coordinates the various specialized teams.
- A research and surveying agent is the data gatherer. It analyzes the building site, researches local zoning laws, and gathers data on material properties (our RAG and API tools). It provides the foundational da needed to build the plan.
- A structural engineer agent is the core synthesizer. It takes the architect’s vision and the researcher’s data and creates the detailed, technical blueprints. It ensures that the design is not just beautiful, but also

Ask

feasible, safe, and efficient. This is our primary “writer” and “reasoner.”

- A compliance review agent is the quality assurance specialist. It meticulously checks the final blueprints against all building codes, safety regulations, and the original client requirements to ensure that there are no errors before construction begins.

Ask